ASSIGNMENT 5: DATABASE CREATION AND TABLE IMPLEMENTATION

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: KUNGUMA KEERTHANA M

REGISTER NO: 2511021240051

COURSE: BCA

YEAR: 2ND YEAR

PROJECT TITLE: Online Food Delivery System

1. Aim

To create a database for the Online Food Delivery System and implement all required tables using SQL DDL commands, ensuring that primary keys, foreign keys, and relationships are properly defined.

2. Software Requirements

- Database Management System: MySQL
- Tool: MySQL Workbench
- Language: SQL (Structured Query Language)

3. Theory

SQL stands for Structured Query Language. It is used to create, manage, and retrieve data from relational databases.

Data Definition Language (DDL) is a category of SQL commands used to define and modify database structures.

Important DDL commands:

- CREATE DATABASE: Creates a new database.
- CREATE TABLE: Creates a table with specified columns and constraints.
- PRIMARY KEY: Uniquely identifies each record.
- FOREIGN KEY: Establishes a relationship between tables.
- NOT NULL: Prevents a column from containing NULL values.
- UNIQUE: Prevents duplicate values in a column or group of columns.

4. Database Implementation Using SQL

The following SQL program creates the database and all ten tables according to the Online Food Delivery System design.

1. Create the database

```sql
CREATE DATABASE online_food_delivery;
USE online_food_delivery;
```

2. Create Customer table

```sql
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Phone VARCHAR(15) NOT NULL,
    Password_Hash VARCHAR(255) NOT NULL
);
```

3. Create Address table

```sql
CREATE TABLE Address (
    Address_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID INT NOT NULL,
    House_No VARCHAR(30),
    Street VARCHAR(100),
    City VARCHAR(50),
    Pincode VARCHAR(10),
    FOREIGN KEY (Customer_ID)
        REFERENCES Customer(Customer_ID)
);
```

4. Create Restaurant table

```sql
CREATE TABLE Restaurant (
    Restaurant_ID INT PRIMARY KEY AUTO_INCREMENT,
    Restaurant_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Location VARCHAR(150),
    Rating DECIMAL(2,1)
);
```

5. Create Category table

```sql
CREATE TABLE Category (
    Category_ID INT PRIMARY KEY AUTO_INCREMENT,
    Category_Name VARCHAR(50) NOT NULL UNIQUE
);
```

6. Create Food_Item table

```sql
CREATE TABLE Food_Item (
    Food_ID INT PRIMARY KEY AUTO_INCREMENT,
    Restaurant_ID INT NOT NULL,
    Category_ID INT NOT NULL,
    Food_Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Availability BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (Restaurant_ID)
        REFERENCES Restaurant(Restaurant_ID),
    FOREIGN KEY (Category_ID)
        REFERENCES Category(Category_ID),
    CHECK (Price >= 0)
);
```

7. Create Orders table

```sql
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID INT NOT NULL,
    Address_ID INT NOT NULL,
    Order_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    Order_Status VARCHAR(30) DEFAULT 'Placed',
    FOREIGN KEY (Customer_ID)
        REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Address_ID)
        REFERENCES Address(Address_ID),
    CHECK (Total_Amount >= 0)
);
```

8. Create Order_Item table

```sql
CREATE TABLE Order_Item (
    Order_Item_ID INT PRIMARY KEY AUTO_INCREMENT,
    Order_ID INT NOT NULL,
    Food_ID INT NOT NULL,
    Quantity INT NOT NULL,
    Subtotal DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID),
    FOREIGN KEY (Food_ID)
        REFERENCES Food_Item(Food_ID),
    CHECK (Quantity > 0),
    CHECK (Subtotal >= 0)
);
```

9. Create Payment table

```sql
CREATE TABLE Payment (
    Payment_ID INT PRIMARY KEY AUTO_INCREMENT,
    Order_ID INT NOT NULL,
    Payment_Method VARCHAR(30) NOT NULL,
    Payment_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Amount DECIMAL(10,2) NOT NULL,
    Payment_Status VARCHAR(30) DEFAULT 'Pending',
    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID),
    CHECK (Amount >= 0)
);
```

10. Create Delivery_Partner table

```sql
CREATE TABLE Delivery_Partner (
    Partner_ID INT PRIMARY KEY AUTO_INCREMENT,
    Partner_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) NOT NULL,
    Vehicle_No VARCHAR(20),
    Status VARCHAR(30) DEFAULT 'Available'
);
```

11. Create Delivery table

```sql
CREATE TABLE Delivery (
    Delivery_ID INT PRIMARY KEY AUTO_INCREMENT,
    Order_ID INT NOT NULL,
    Partner_ID INT NOT NULL,
    Pickup_Time DATETIME,
    Delivery_Time DATETIME,
    Delivery_Status VARCHAR(30) DEFAULT 'Assigned',
    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID),
    FOREIGN KEY (Partner_ID)
        REFERENCES Delivery_Partner(Partner_ID)
);
```

5. Relationships Between Tables

Parent table	Child table	Relationship
Customer	Address	One-to-many
Customer	Orders	One-to-many
Address	Orders	One-to-many
Restaurant	Food_Item	One-to-many
Category	Food_Item	One-to-many
Orders	Order_Item	One-to-many
Food_Item	Order_Item	One-to-many
Orders	Payment	One-to-many in this implementation
Orders	Delivery	One-to-many in this implementation
Delivery_Partner	Delivery	One-to-many

Foreign keys maintain referential integrity by ensuring that referenced records exist in the parent tables.

6. Commands to Verify Table Creation

After executing the SQL program, run the following commands to verify the database and its tables.

1. Display the selected database

Input:

```
SELECT DATABASE();
```

Output:

DATABASE()
online_food_delivery

2. Display all tables

Input:

```
SHOW TABLES;
```

Output:

Tables_in_online_food_delivery
address
category
customer
delivery
delivery_partner
food_item
order_item
orders
payment
restaurant

3. View the Customer table structure

Input:

```
DESCRIBE Customer;
```

Output:

Field	Type	Null	Key	Default	Extra
Customer_ID	int	NO	PRI	NULL	auto_increment
Name	varchar(100)	NO		NULL	
Email	varchar(100)	NO	UNI	NULL	
Phone	varchar(15)	NO		NULL	
Password_Hash	varchar(255)	NO		NULL	

4. View the Food_Item table structure

Input:

```
107 •    DESCRIBE Food_Item;
```

Output:

Field	Type	Null	Key	Default	Extra
Food_ID	int	NO	PRI	NULL	auto_increment
Restaurant_ID	int	NO	MUL	NULL	
Category_ID	int	NO	MUL	NULL	
Food_Name	varchar(100)	NO		NULL	
Price	decimal(10,2)	NO		NULL	
Availability	tinyint(1)	YES		1	

5. View the Orders table structure

Input:

```
111 •    SHOW CREATE TABLE Order_Item;
```

Output:

Table	Create Table
Order_Item	CREATE TABLE `order_item` (`Order_Item_I...

The SHOW TABLES command should display all ten tables created in the database.

7. Procedure

1. Open MySQL Workbench and connect to the MySQL server.
2. Create a new SQL query tab.
3. Copy the SQL program into the query editor.
4. Execute the complete script in the given order.
5. Check for errors in the output panel.
6. Run SHOW TABLES; to verify the creation of all tables.
7. Run DESCRIBE commands to inspect the table structures.
8. Save the SQL script and capture screenshots of the successful execution for the assignment report.

8. Result

The Online_Food_Delivery database was defined using SQL DDL commands. Ten tables were implemented with appropriate attributes, primary keys, foreign keys, and constraints to establish the required relationships.

9. Conclusion

The Online Food Delivery System database structure was implemented using MySQL. Primary keys provide unique identification, while foreign keys connect related tables and maintain referential integrity. This implementation provides a foundation for storing and managing customer, restaurant, food item, order, payment, and delivery information.